**Name:** _____

**Class:** _____

**Mr. Rawson • GCSE Computer Science** Year 11 2026

# Algorithms 4 — Sorting: Bubble, Insertion and Merge

Last lesson ended on one rule: **binary search needs a sorted list**. Putting a list in order is a job of its own, and OCR names three standard ways to do it. In the exam you **apply** each one to a list of data and show every stage, so this sheet is all stepping and tracing.

**OCR J277 specification, 2.1.3 Searching and sorting algorithms**

**Required**

- ✓ Understand the main steps of the algorithm and the segments of code in it
- ✓ Understand any pre-requisites of an algorithm
- ✓ Apply the algorithm to a data set
- ✓ Identify an algorithm if given the code, pseudocode or Exam Reference Language for it

**Not required**

- × To remember the code, pseudocode or Exam Reference Language for these algorithms

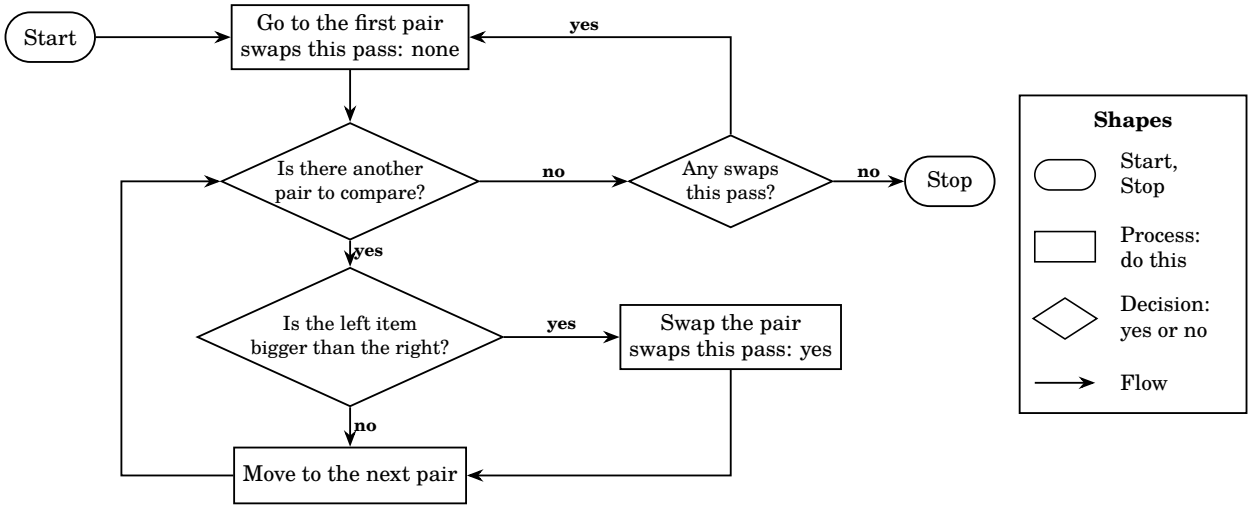
## 1 Sort it by hand first

Write each of these numbers on its own scrap of paper, mix them up, and put them in ascending order on the desk:

7 3 9 2 6 4

**Now try •** Finish the sentence that describes **your method**: "I started by … and then I …"

**The same job as a flowchart.** Every box is something your hands did. A **pass** is one trip along the list; the algorithm repeats passes until one pass needs no swaps.



## 2 Bubble sort

**VOCAB** **Bubble sort** (OCR 2.1.3) — compares each pair of neighbouring items from the start of the list and swaps the pair when it is in the wrong order, then repeats the whole pass until a pass makes no swaps

**Watch it run:** `rawsonvault.com/cs/gcse/2-1-algorithms/sorting-animations/bubble-sort.html`

**The steps.**

1. **Compare**: Start at the first item. Compare it with its neighbour on the right.
2. **Swap**: If the left item is **bigger**, swap the two. If not, leave them.
3. **Finish the pass**: Carry on to the end of the list: that is one **pass**.
4. **Count the swaps**: Count the swaps in the pass. If there was at least one, start another pass from the left. If there were **none**, the list is sorted: stop.

**Worked example, pass 1.** The list is `7 3 9 2 6 4`. Five pairs are compared. At each comparison the pair is outlined; the row below shows the list after it.

| | | |
|---|---|---|
| **Compare 1** | **7** **3** 9 2 6 4 | 7 is bigger than 3: **swap** |
| **Compare 2** | 3 **7** **9** 2 6 4 | 7 is not bigger than 9: **no swap** |
| **Compare 3** | 3 7 **9** **2** 6 4 | 9 is bigger than 2: **swap** |
| **Compare 4** | 3 7 2 **9** **6** 4 | 9 is bigger than 6: **swap** |
| **Compare 5** | 3 7 2 6 **9** **4** | 9 is bigger than 4: **swap** |
| **End of pass 1** | 3 7 2 6 4 **9** | 4 swaps: another pass; 9 is in place |

Pass 1 made 4 swaps, so another pass is needed. The biggest item, 9, has moved right until it reached the end: after every pass the next biggest item is in its final place.

**How a swap works.** Two boxes cannot swap contents directly, so a program uses a third box, `temp`: copy the left item into `temp`, copy the right item over the left, then copy `temp` into the right.

**Now try** • Complete the pass table. Each row is one whole pass, starting from the list at the end of the pass before. Stop on the pass that makes no swaps.

| Pass | List at the end of the pass | Swaps | Another pass? |
|---|---|---|---|
| 1 | 3 7 2 6 4 9 | 4 | yes, there were swaps |
| 2 | | | |
| 3 | | | |
| 4 | | | |

**The same algorithm as code.** The exam shows it in OCR ERL; you can type the Python beside it and run it. The `swapped` flag is what records whether a pass made any swap.

**OCR ERL (what the exam shows)**

```
swapped = true
while swapped
  swapped = false
  for i = 0 to numbers.length - 2
    if numbers[i] > numbers[i + 1] then
      temp = numbers[i]
      numbers[i] = numbers[i + 1]
      numbers[i + 1] = temp
      swapped = true
    endif
  next i
endwhile
```

**PYTHON (what you type)**

```
def bubble_sort(items):
    swapped = True
    while swapped:
        swapped = False
        for i in range(len(items) - 1):
            if items[i] > items[i + 1]:
                temp = items[i]
                items[i] = items[i + 1]
                items[i + 1] = temp
                swapped = True
        print(items)

bubble_sort([7, 3, 9, 2, 6, 4])
```

**Now try** • Type the Python and run it. Then match each job to its ERL line number.

| The line that … | ERL line |
|---|---|
| compares a pair of neighbours | |
| copies the left item into the spare box | |
| records that this pass made a swap | |
| keeps the passes going while the last pass had a swap | |

**The questions you will meet on bubble sort.** Every bubble sort question in the open past-paper pool asks for one of these five things.

| The question asks you to | What to write | Marks |
|---|---|---|
| Show the stages of a bubble sort on a list | one row per pass (or per swap), writing the whole list each time | 4 |
| Explain the purpose of the Boolean variable | it records whether a swap has taken place in the pass; the `while` line checks it to decide whether to go round again | 2 |
| Explain the lines that swap two items | they swap `items[p]` and `items[p + 1]` when the left one is bigger, using `temp` so that neither value is lost | 2 |
| Name two other sorting algorithms | insertion sort and merge sort | 2 |
| Give an advantage of merge sort over bubble sort | it is faster on long lists, and its running time does not depend on how ordered the list was | 2 |

**Worked example.** This one is written for this sheet, in the style of the exam question that follows it. The answer is shown, so you can see what a full answer looks like.

*Show the stages of a bubble sort when applied to this list of words, A to Z.*

`mango` `apple` `zebra` `lemon` `grape` `kiwi`

**Method.** Write the starting list. Take one pass from left to right, swapping each pair where the left word comes later in the alphabet. Write the *whole* list at the end of the pass. Go round again, and stop after a pass that makes no swaps.

| **Stage** | **The whole list** | | | | | | **Swaps** |
|---|---|---|---|---|---|---|---|
| start | `mango` | `apple` | `zebra` | `lemon` | `grape` | `kiwi` | |
| pass 1 | `apple` | `mango` | `lemon` | `grape` | `kiwi` | `zebra` | 4 |
| pass 2 | `apple` | `lemon` | `grape` | `kiwi` | `mango` | `zebra` | 3 |
| pass 3 | `apple` | `grape` | `kiwi` | `lemon` | `mango` | `zebra` | 2 |
| pass 4 | `apple` | `grape` | `kiwi` | `lemon` | `mango` | `zebra` | 0 |

Pass 4 makes no swaps, so the list is sorted and the answer ends there. After pass 1 the last word, zebra, is already in its final place; after pass 2, mango is too. In pass 1, `mango` is compared with `apple` and swaps; then with `zebra`, and stays; `zebra` then swaps past `lemon`, `grape` and `kiwi`.

*The real exam questions for this section are on the printed copy used in class.*

## 3 Insertion sort

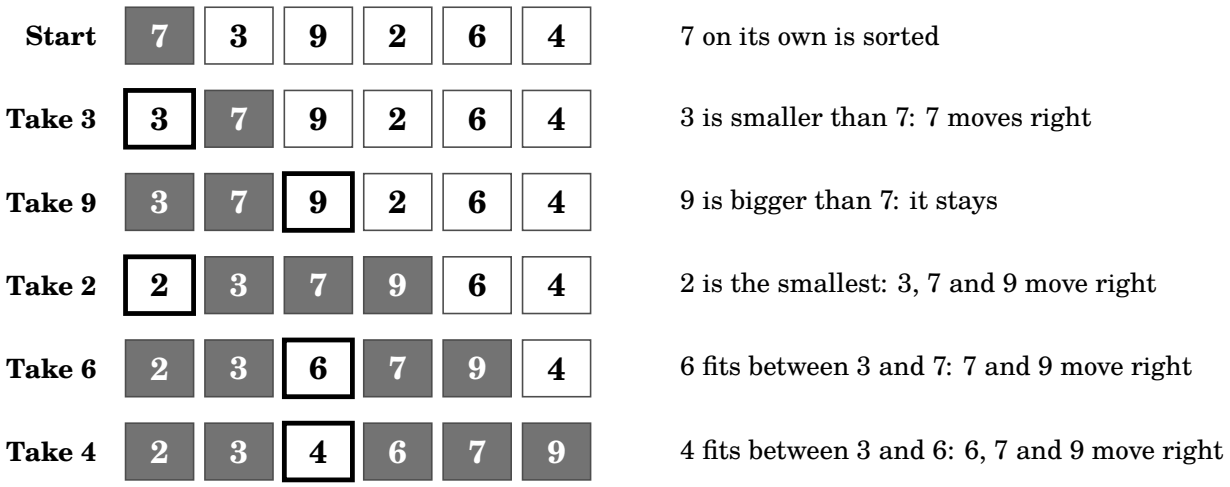
**VOCAB** **Insertion sort** (OCR 2.1.3) — splits the list into a sorted part on the left and an unsorted part on the right, then takes the unsorted items one at a time and inserts each into its correct place in the sorted part

**Watch it run:** `rawsonvault.com/cs/gcse/2-1-algorithms/sorting-animations/insertion-sort.html`

**The steps.**

1. **Start**: The **first item** on its own is the sorted part. Everything else is unsorted.
2. **Compare**: Take the first unsorted item. Compare it with the sorted items, working leftwards from the biggest, until you find where it belongs.
3. **Insert**: Insert it there. The bigger sorted items each move one place right.
4. **Repeat**: The sorted part is now one item longer. Repeat until the unsorted part is empty.

**Worked example.** The same list, `7 3 9 2 6 4`. The dark cards are the **sorted part**. The white card with the heavy outline is the item just inserted.




**The same algorithm as code.** The strips above are this program running on `7 3 9 2 6 4`. Type the Python and run it: it prints the list after each card is put down, the same five lists as the strip.

**PYTHON (what you type)**

```
def insertion_sort(items):
    for index in range(1, len(items)):
        item_to_insert = items[index]
        position = index - 1
        while position >= 0 and items[position] > item_to_insert:
            items[position + 1] = items[position]
            position = position - 1
        items[position + 1] = item_to_insert
        print(items)

insertion_sort([7, 3, 9, 2, 6, 4])
```

**OCR ERL (what the exam shows)**

```
1 for index = 1 to items.length - 1
2   itemToInsert = items[index]
3   position = index - 1
4   while position >= 0 AND items[position] > itemToInsert
5     items[position + 1] = items[position]
6     position = position - 1
7   endwhile
8   items[position + 1] = itemToInsert
9 next index
```

| Python line | What it does on the cards |
|---|---|
| 2 | takes the next card from the unsorted part; `index` is its place |
| 3–4 | holds that card in hand, and points at the sorted card just to its left |
| 5 | asks whether that sorted card is bigger than the card in hand (and that a sorted card is still there) |
| 6–7 | slides the bigger sorted card one place right, then points at the next sorted card to the left |
| 8 | puts the card in hand down in the gap that is left |

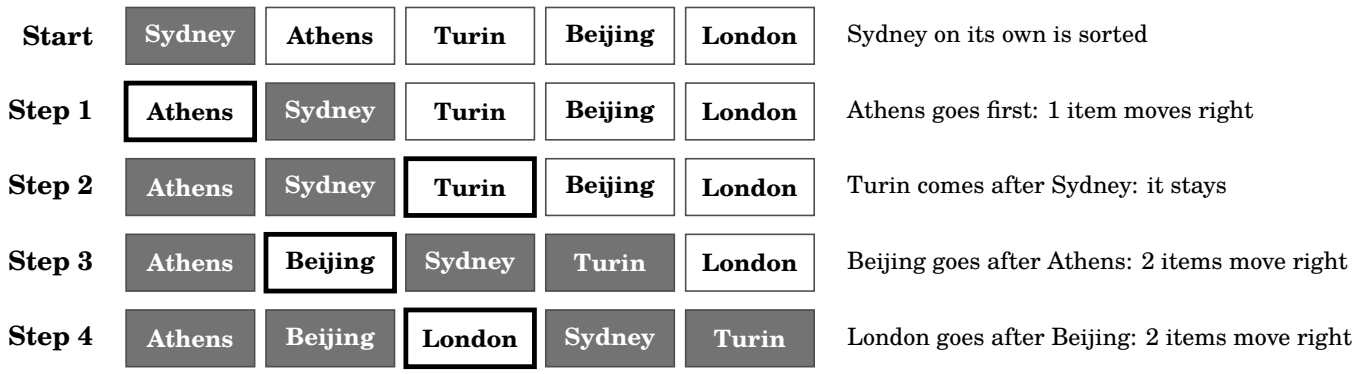
**Worked example 2: seven cards.** The same steps on seven cards. Dark cards are the sorted part; the heavy outline is the card just inserted.

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **Start** | 9 | 3 | 10 | 2 | 8 | 5 | 1 | 9 on its own is sorted |
| **Take 3** | 3 | 9 | 10 | 2 | 8 | 5 | 1 | 3 is the smallest so far: 1 item moves right |
| **Take 10** | 3 | 9 | 10 | 2 | 8 | 5 | 1 | 10 is bigger than 9: it stays |
| **Take 2** | 2 | 3 | 9 | 10 | 8 | 5 | 1 | 2 is the smallest so far: 3 items move right |
| **Take 8** | 2 | 3 | 8 | 9 | 10 | 5 | 1 | 8 fits after 3: 2 items move right |
| **Take 5** | 2 | 3 | 5 | 8 | 9 | 10 | 1 | 5 fits after 3: 3 items move right |
| **Take 1** | 1 | 2 | 3 | 5 | 8 | 9 | 10 | 1 is the smallest so far: 6 items move right |

**Worked example 3: medals.** The number of medals won by five countries at the 1998 Winter Olympics.

| | | | | | | |
|---|---|---|---|---|---|---|
| **Start** | 15 | 25 | 13 | 29 | 18 | 15 on its own is sorted |
| **Take 25** | 15 | 25 | 13 | 29 | 18 | 25 is bigger than 15: it stays |
| **Take 13** | 13 | 15 | 25 | 29 | 18 | 13 is the smallest so far: 2 items move right |
| **Take 29** | 13 | 15 | 25 | 29 | 18 | 29 is bigger than 25: it stays |
| **Take 18** | 13 | 15 | 18 | 25 | 29 | 18 fits after 15: 2 items move right |

**Worked example 4: words.** Words are put in alphabetical order, A to Z.




Examples 2 to 4 use the data from the insertion sort page of Isaac Computer Science.

**Now try** • Insertion sort a new list, `5 1 4 8 2 7`. Take the items in turn and write the whole list on each row, with the bar in its new place.

| Item taken | Sorted part \| Unsorted part | What happened |
|---|---|---|
| (start) | 5 \| 1 4 8 2 7 | 5 on its own is sorted |
| 1 | | |
| 4 | | |
| 8 | | |
| 2 | | |
| 7 | | |

**Write it: insertion sort in Python.** The outer `for` line starts at index 1, because the item at index 0 is the sorted part to begin with. `index` is the item to insert, and `position` walks left through the sorted part. The `while` line tests `position >= 0` first, so `items[position]` is only read when `position` names a real item.

**Now try • Task 1.** Fill each gap so that the function sorts the list.

```
def insertion_sort(items):
    for index in range(1, ____________):
        item_to_insert = items[index]
        position = ____________
        while position >= ___ and items[position] > _____________:
            items[position + 1] = _____________
            position = position - 1
        items[____________] = item_to_insert
        print(items)
```

**Now try • Task 2.** Run `insertion_sort([6, 2, 8, 3, 1])` in your head. Write the list that `print(items)` shows after each item is inserted, and the number of items that were copied up one place each time.

| Item inserted | Output of `print(items)` | Items copied up |
|---|---|---|
| 2 | | |
| 8 | | |
| 3 | | |
| 1 | | |

**Now try • Task 3 (stretch).** The list is already in ascending order, `[1, 2, 3, 4, 5]`. How many items does the insertion sort copy up in total, and how many times does the `while` loop body run?

Items copied up in total: _____ Times the `while` body runs: _____

## 4 Merge sort

**VOCAB** **Merge sort** (OCR 2.1.3) — a divide and conquer algorithm: it splits the list in halves until every list holds one item, then merges the lists back together in pairs, putting the items in order as it merges

**Watch it run:** `rawsonvault.com/cs/gcse/2-1-algorithms/sorting-animations/merge-sort.html`

**The steps.**

1. **Divide**: Split the list into single items. A list of one item is already sorted.
2. **Merge**: Take two neighbouring lists. Compare their **first** items, move the smaller into the new list, and repeat until one list is empty. Add whatever is left of the other list on the end.
3. **Keep merging**: Repeat step 2 with the new lists until a single list remains.

**Worked example, one merge.** Merge `3 7` with `2 9`.

| Step | Left list | Right list | Smaller first item | New list |
|---|---|---|---|---|
| 1 | 3 7 | 2 9 | 2 | 2 |
| 2 | 3 7 | 9 | 3 | 2 3 |
| 3 | 7 | 9 | 7 | 2 3 7 |
| 4 | (empty) | 9 | 9 is left over | 2 3 7 9 |

**Worked example, the whole sort.** Eight items, divided into single items, then merged back in three rounds.

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **Lists of 1** | 7 | 3 | 9 | 2 | 6 | 4 | 8 | 1 |
| **Lists of 2** | 3 7 | | 2 9 | | 4 6 | | 1 8 | |
| **Lists of 4** | 2 3 7 9 | | | | 1 4 6 8 | | | |
| **Lists of 8** | 1 2 3 4 6 7 8 9 | | | | | | | |

Each merge produces a list that is already in order, so no list is ever sorted again afterwards: the merging *is* the sorting.

**Now try** • Merge sort this list. The row of single items is done for you. Fill in the three merge rounds, drawing each new list across the boxes it covers.

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **Lists of 1** | 5 | 1 | 8 | 3 | 7 | 2 | 6 | 4 |
| **Lists of 2** | | | | | | | | |
| **Lists of 4** | | | | | | | | |
| **Lists of 8** | | | | | | | | |

**The same algorithm as code.** Merge sort in Python is two functions. `merge` joins two sorted lists into one. `merge_sort` splits the list in half, calls itself on each half, and merges the two results. Type it and run it on the list from your worked example: the exam asks you to follow merge sort on a list of data, and the program shows every merge it makes.

**PYTHON (what you type)**

```
1  def merge(left, right):
2      merged = []
3      i = 0
4      j = 0
5      while i < len(left) and j < len(right):
6          if left[i] <= right[j]:
7              merged.append(left[i])
8              i = i + 1
9          else:
10             merged.append(right[j])
11             j = j + 1
12     merged = merged + left[i:] + right[j:]
13     return merged
14
15 def merge_sort(items):
16     if len(items) <= 1:
17         return items
18     middle = (len(items) + 1) // 2
19     left = merge_sort(items[:middle])
20     right = merge_sort(items[middle:])
21     merged = merge(left, right)
22     print(left, right, "->", merged)
23     return merged
24
25 merge_sort([7, 3, 9, 2, 6, 4, 8, 1])
```

**What it prints.** One line for each merge, in the order the program does them.

```
[7] [3] -> [3, 7]
[9] [2] -> [2, 9]
[3, 7] [2, 9] -> [2, 3, 7, 9]
[6] [4] -> [4, 6]
[8] [1] -> [1, 8]
[4, 6] [1, 8] -> [1, 4, 6, 8]
[2, 3, 7, 9] [1, 4, 6, 8] -> [1, 2, 3, 4, 6, 7, 8, 9]
```

| Python line | What it does |
|---|---|
| 1–13 | `merge`: the merge step from the table above, as a function |
| 16–17 | a list of one item, or none, is already sorted, so it is handed back as it is |
| 18 | finds the middle; with an odd number of items the first half takes the extra one |
| 19–20 | sorts each half by calling `merge_sort` on it |
| 21 | merges the two sorted halves |
| 22 | prints the merge, so the lists of 2, then 4, then 8 can be read off |

**Write it: the merge step in Python.** The merge is the part of merge sort that does the sorting, and it is short. `i` marks the first unused item of `left` and `j` the first unused item of `right`. Each trip round the loop moves the smaller of those two items into `merged`. When one list runs out, the rest of the other list goes on the end, already in order.

**Now try • Task 1.** Fill each gap so that `merge` joins two sorted lists into one sorted list.

```
def merge(left, right):
    merged = []
    i = 0
    j = 0
    while i < len(left) and j < ____________:
        if left[i] <= ____________:
            merged.append(left[i])
            i = i + 1
        else:
            merged.append(____________)
            j = j + ___
    merged = merged + left[i:] + ____________
    return merged
```

**Now try • Task 2.** Run `merge([2, 5, 9], [3, 4])` in your head. Write `merged` after each comparison, then after the leftover items are added.

| Step | `merged` |
|---|---|
| Compare 2 and 3 | |
| Compare 5 and 3 | |
| Compare 5 and 4 | |
| Add the leftover items | |

**Now try • Task 3 (stretch).** A merge sort splits `[15, 25, 13, 29, 18]` in half. With an odd number of items the first half takes the extra one. Write the two halves.

First half: _____ Second half: _____

## 5 Three sorts side by side

| | Bubble sort | Insertion sort | Merge sort |
|---|---|---|---|
| The idea | swap neighbours that are the wrong way round, pass after pass | grow a sorted part on the left, one inserted item at a time | divide into single items, then merge in order |
| What stops it | a pass with no swaps | the unsorted part is empty | one list is left |
| The list at the start | any order | any order | any order |
| On a long list | slowest of the three | faster than bubble sort | fastest of the three |

**Now try** • Name the sort each statement describes: **bubble**, **insertion** or **merge**.

| | Statement | Sort |
|---|---|---|
| a | The list is split into single items before anything is compared. | |
| b | Each pass compares neighbouring pairs from the left of the list. | |
| c | The first item is treated as a sorted list of one. | |
| d | It stops when a pass completes without a swap. | |
| e | Each new item is slotted into its place among the items already in order. | |
| f | Two ordered lists are combined by repeatedly taking the smaller first item. | |

**Sort first, then search.** A sorted list is what binary search was waiting for. Sort `7 3 9 2 6 4` with any of the three sorts, and `2 3 4 6 7 9` is ready for it.

**Now try** • Use binary search on the sorted list `2 3 4 6 7 9` to find **9**. List each middle item you check.

## The exam answers, one line each

| Asked for | Write |
|---|---|
| Show the stages of a bubble sort | One row per pass (or per swap), writing the whole list each time, the biggest remaining item moving to the right. |
| Show the stages of an insertion sort | One row per inserted item, the sorted part growing by one each row. |
| Show the stages of a merge sort | Split into single items, then merge into lists of 2, then 4, then the whole list, each list in order. |
| An advantage of merge sort | It is faster on long lists, and its running time does not depend on how ordered the list was to begin with. |